Agentic RAG & LangGraph

A Study Guide for Data Scientist Interviews

Retrieval-Augmented Generation • Embeddings • Vector Search • Agents • LangGraph • Evaluation

Built from the **Agentic RAG Research Assistant** portfolio project — question-answering over 2,012 arXiv papers, with a from-scratch agent and a LangGraph rebuild.

Prepared by Cena Shabani • Companion to the project's `learning/` course

How to use this guide

This is an interview-prep companion to a real project, not an abstract textbook. Every concept here is something the project implements, so you can always point to code when an interviewer pushes for depth. The through-line is one honest, defensible story:

> **The finding to quote**
>
> Single-shot retrieval nails single-hop questions (recall@5 = 1.00) but collapses to 0.60 on multi-hop ones, because one search cannot cover two pieces of evidence. That gap is the entire reason to build an **agent** that retrieves iteratively. Being able to state a measured finding like this — with the number and the mechanism — is what separates 'I did a RAG tutorial' from 'I can reason about RAG'.

Each chapter ends with the interview-relevant takeaway. The last two chapters are a **Q&A bank** of questions you are likely to be asked, and a **one-page cheat-sheet** to review the morning of an interview.

Contents

#	Chapter	The one idea
1	What RAG is and why it exists	Give the model an open-book exam
2	Embeddings & semantic search	Turn meaning into geometry
3	Chunking	One vector can only mean one thing
4	Vector stores & ANN search	Fast nearest-neighbour at scale
5	The RAG pipeline end to end	Build-time vs query-time
6	Prompting & grounding	Answer from the sources, and cite
7	Agentic RAG	Let the model decide how to retrieve
8	Agents, tools & the loop	Reason, act, observe, repeat
9	LangGraph	Control flow as a drawable graph
10	Evaluating RAG	Measure retrieval and answers separately
11	Failure modes & limits	Where RAG breaks, honestly
12	Interview Q&A bank	40+ questions with answers
13	One-page cheat-sheet	Review this the morning of

1 · What RAG is and why it exists

Retrieval-Augmented Generation (RAG): before a language model answers, fetch the relevant documents and place them in the prompt, so the model answers *from those documents* instead of from memory.

The exam analogy

A plain LLM answering from memory is a student sitting a **closed-book** exam — fluent, and confidently wrong on specifics. RAG makes it **open-book**: for each question it flips to the relevant pages, reads them, and answers with them open. The model's job shifts from *recall* to *reading comprehension* — a far more reliable task.

The three problems RAG fixes

Problem	Bare LLM	With RAG
Knowledge is frozen at training	Cannot know new facts	Add documents to the corpus
Context window is finite	Cannot read the whole corpus	Retrieve only what is relevant
Hallucination	Invents a fluent, wrong answer	Answers from sources, or abstains

Crucially, all three are **retrieval** fixes, not **model** fixes. You often don't need a bigger or newer model — you need to give the model you have the right context.

> **Interview tip**
>
> If asked 'why not just fine-tune?' answer: fine-tuning teaches *style and behaviour*, not fresh facts; it is expensive, slow to update, and still hallucinates. RAG updates knowledge by editing a document store — cheap and traceable. They solve different problems and are often combined.

2 · Embeddings & semantic search

Keyword matching breaks the moment a document says the same thing in different words. **Embeddings** let us match on meaning. An embedding model maps text to a vector — a point in a high-dimensional space — such that texts with similar meaning land near each other.

Search becomes geometry

- Embed every passage in the corpus → a cloud of points.
- Embed the query into the *same* space → one more point.
- Return the passages whose points are **nearest** the query point.

No shared keywords required — if two texts *mean* the same thing, their points are close.

Cosine similarity and the normalization trick

'Near' is measured by **cosine similarity** — the cosine of the angle between two vectors, from -1 (opposite) to 1 (identical direction). If you normalize every vector to unit length, the plain **dot product equals cosine similarity** — which lets a fast dot-product index mean 'most similar in meaning'. This project uses `all-MiniLM-L6-v2` (384-dim, CPU, deterministic).

```
vectors = model.encode(texts, normalize_embeddings=True)  # unit vectors
# now: dot(a, b) == cosine_similarity(a, b)
```

> **Interview tip**
>
> Know the difference between a **bi-encoder** (embeds query and document separately — fast, used for first-stage retrieval) and a **cross-encoder** (reads query+document together — slower, more accurate, used for *reranking* the top results).

3 · Chunking

An embedding model compresses its whole input into **one** vector. Embed an entire paper and that vector is an average of every idea in it — near nothing in particular. So we split documents into **chunks** and embed each separately. Precision comes from each chunk meaning one thing.

The Goldilocks size, and overlap

- **Too big** → blurry embeddings that match nothing precisely.
- **Too small** → a sentence loses the context that made it meaningful.
- **Sweet spot** → a few sentences / one paragraph (this project: ~1000 chars).
- **Overlap** (~15%) → consecutive chunks share text, so a fact that straddles a boundary survives intact in at least one chunk.

Chunks must also carry **metadata** (source id, title) so an answer can *cite* the passage it used — no metadata, no citations, no auditability.

> **Interview tip**
>
> Good follow-up to raise yourself: **semantic** or **recursive** chunking (split on sentence/section boundaries) beats fixed-size character windows on structured documents. Mentioning the upgrade path shows you know the default has limits.

4 · Vector stores & ANN search

A **vector store** holds the chunk vectors and finds the nearest ones to a query fast.

Approach	How	When
Exact (flat)	Compare query to every vector	Small corpus (this project: 4,298 chunks) — instant, no approxin

Approximate (ANN)	HNSW graphs, IVF, PQ — search a fraction of vectors	Millions+ of vectors — trade a little recall for big speed

This project uses FAISS `IndexFlatIP` (exact, inner-product). Because search is exact, retrieval quality reflects the embeddings and chunking — with no ANN approximation muddying the evaluation. The index stores vectors and returns *row numbers*; a row-aligned metadata table maps them back to chunk text and citations.

> **Interview tip**
> 'How would this scale to 100M documents?' → switch the flat index for an ANN index (HNSW), move to a hosted vector DB (pgvector / Pinecone / Weaviate), and add a reranker. The *concepts* are identical; you're changing the index type and the ops, not the pipeline.

5 · The RAG pipeline end to end

```
ACQUIRE -> CHUNK -> EMBED -> INDEX   ||   RETRIEVE -> GENERATE
\__________ build time __________/         \___ query time ___/
```

The single most clarifying idea in RAG is the **build-time vs query-time** split:

- **Build time** (run once): turn documents into a searchable index. Expensive work — embedding the whole corpus — paid once.
- **Query time** (per question): embed one query, search, generate. Cheap.

Two consequences fall out: updating knowledge is cheap (re-index, no retraining), and retrieval is local/reproducible while generation is the only hosted part. Use the **same embedding model** at build and query time — the query must live in the same geometry as the chunks.

> **Interview tip**
> Emphasise the **single source of truth**: both the plain and agentic paths call the same retriever, so any measured difference between them is about the *strategy*, never an accidental difference in search. Controlling variables is a data-scientist instinct interviewers look for.

6 · Prompting & grounding

Grounding = binding the answer to the evidence. It is enforced by three instructions in the system prompt:

- **'Answer using ONLY the provided passages.'** Shifts the model from recall to reading comprehension — the core anti-hallucination lever.
- **'Cite the source ids you used.'** Makes every claim auditable and nudges the model to actually use the passages.
- **'If the passages are insufficient, say so.'** Makes 'I don't know' a correct answer — permission to abstain is a hallucination guardrail.

We also verify citations: a ‘citation’ to an id that was never retrieved is a hallucinated reference and is not credited. Grounding *reduces* hallucination; it does not abolish it (the model can misread, or retrieval can hand it the wrong passage) — which is why we *measure* groundedness rather than assume it.

7 · Agentic RAG

Plain RAG retrieves **once**, up front, and hopes that single search grabbed everything. **Agentic RAG** puts the LLM in charge of retrieval: it can decompose the question, search for each part, read the results, decide whether it has enough, and search again before answering.

> **The finding to quote**
>
> This matters because of the measured multi-hop gap. A question like ‘how does RAG combine a retriever with a generator?’ needs evidence from TWO papers; one search lands in one region of the vector space and misses the second. The agent’s second, reformulated search is what recovers it — turning 0.60 recall into a real answer.

The trade-off — say this out loud

Agentic RAG is not free: multiple LLM calls and retrievals per question means more tokens, latency, and cost. The honest framing is a measured exchange rate — *better multi-hop answers at N× the retrieval cost* — not ‘the fancy version is better’. Interviewers reward the candidate who names the cost, not just the benefit.

8 · Agents, tools & the loop

An **agent** is an LLM that uses **tools** in a **loop** toward a goal. A tool is a function described to the model (name, description, input schema); the model emits a request to call it, your code runs it, and you feed the result back. This project’s one tool is `search_corpus(query)`.

The tool-use loop (ReAct)

```
send conversation + tools to the LLM
loop:
  if the reply is a tool call  -> run the tool, append the result, repeat
  if the reply is plain text   -> that's the answer, stop
  (bounded by MAX_AGENT_STEPS so a confused agent can't loop forever)
```

ReAct = Reason + Act: the model interleaves reasoning with tool actions. The bounded loop and a ‘stopped reason’ (answered vs hit-the-cap) are what make it production-safe.

> **Interview tip**
>
> Be ready to contrast building the loop **from scratch** (maximally transparent — every decision is an inspectable message) with a framework like **LangGraph** (next chapter), which you reach for when the loop needs checkpointing, streaming, or human-in-the-loop.

9 · LangGraph

LangGraph models an agent as a directed graph / state machine. Instead of writing `if`/`while` control flow by hand, you *declare* a graph and the runtime walks it. Four primitives:

Primitive	What it is	Plain-Python analogue
State	A shared TypedDict every step reads/writes	the variables in your loop
Node	A function: state -> partial state update	one step of the loop body
Edge	'after A, go to B'	statement order
Conditional edge	A router that reads state and picks the next node	an if / while branch

You add nodes and edges to a `StateGraph`, then `.compile()` it into a runnable and `.invoke()` or `.stream()` it. A key detail: a state channel annotated with a **reducer** (`Annotated[list, operator.add]`) *accumulates* instead of overwriting — perfect for gathering documents across retrieval loops.

Our agent as a graph

```
START -> plan -> retrieve -> decide --(need more)--> plan
                             decide --(enough)-----> generate -> END
```

`plan` proposes the next focused query; `retrieve` runs the shared retriever and appends hits; `decide` is the conditional edge (enough distinct papers, or hit the cap?); `generate` writes the grounded answer. Same behaviour as the from-scratch loop — now a graph you can draw, checkpoint, and stream. LangGraph also ships `create_react_agent`, a one-liner for the standard tool-calling loop.

> **Interview tip**
>
> The map to memorise: hand-written `while` ↔ compiled StateGraph; the `if` that decides 'search again?' ↔ a conditional edge; the de-duped evidence dict ↔ a reduced state channel; hand-written tool JSON ↔ the `@tool` decorator.

10 · Evaluating RAG

A RAG system you can't measure is a demo. The most important idea: **evaluate retrieval and generation separately** — they fail for different reasons and are fixed in different places.

Retrieval metrics (need ground-truth relevant docs)

Metric	Question it answers	Definition
hit@k	Did any relevant doc appear in top-k?	1 if intersection non-empty
recall@k	What fraction of relevant docs were retrieved?	|retrieved & relevant| / |relevant|
MRR	How high was the first relevant doc ranked?	1 / rank of first relevant

Answer metrics (LLM-as-judge)

Answer quality has no single correct string, so use a separate LLM call to grade against a rubric — **groundedness** (is every claim supported by the sources?) and **correctness** (does it answer the question?). An answer can be grounded but wrong, or correct but ungrounded, so you measure both. LLM-as-judge is a good *relative* comparator (same judge, same rubric), a weaker *absolute* score.

> **The finding to quote**
>
> The fair comparison holds everything constant except the strategy — same corpus, retriever, judge, and questions — and slices by hop type, because the hypothesis is specific: the agent wins on multi-hop, roughly ties on single-hop, and costs more either way. Report benefit AND cost.

11 · Failure modes & limits

- **Garbage retrieval → garbage answer.** Retrieval is the ceiling on answer quality; no prompt fixes wrong passages.
- **Bad chunking.** Wrong-sized or mid-idea chunks quietly wreck retrieval.
- **Lost in the middle.** LLMs attend most to the start/end of context and skim the middle — so keep top-k small and rank the best first.
- **Multi-hop weakness.** Single-shot retrieval misses compound evidence (the project's finding); the agent fixes it at a token cost.
- **Cost & latency.** Every answer is ≥1 LLM call; agents are several. Mitigate with local embeddings, caching, a bounded loop, a cheaper model.
- **Prompt injection.** Retrieved text is untrusted — a document saying 'ignore your instructions' is *indirect prompt injection*. Treat retrieved content as data, never as instructions.
- **Corpus boundary.** RAG can only answer from what's in the corpus; the correct behaviour outside it is to abstain.

> **Interview tip**
>
> Naming what a project **deliberately does not do** (no reranker, no fine-tuning, no production vector DB, not hardened against injection) reads as maturity, not weakness. Scope honesty is a senior signal.

12 · Interview Q&A bank

Concise, defensible answers. Say the core sentence, then one supporting detail.

Fundamentals

Q. What is RAG, in one sentence?

A. Retrieve relevant documents for a query and put them in the prompt so the LLM answers from them rather than from memory — an open-book exam instead of a closed-book one.

Q. RAG vs fine-tuning?

A. RAG injects fresh, traceable knowledge at query time and is cheap to update; fine-tuning changes the model's style/behaviour, is costly, and still hallucinates facts. Different jobs, often combined.

Q. Why not just put everything in a long context window?

A. Cost, latency, and quality: huge contexts are expensive and trigger 'lost in the middle'. Retrieval is a filter that shows the model only what matters.

Embeddings & retrieval

Q. What is an embedding?

A. A vector representation of text where semantic similarity becomes geometric closeness, so search can match meaning instead of keywords.

Q. Why normalize embeddings?

A. Unit-length vectors make the dot product equal cosine similarity, so a fast inner-product index means 'most similar in meaning' with no length bias.

Q. Bi-encoder vs cross-encoder?

A. Bi-encoder embeds query and doc separately (fast, first-stage retrieval); cross-encoder reads them together (slow, accurate, used to rerank the top candidates).

Q. Exact vs approximate nearest-neighbour search?

A. Exact compares against every vector (fine for small corpora, no approximation error); ANN (HNSW/IVF/PQ) searches a fraction for big speed-ups at a small recall cost — a scale decision.

Chunking & pipeline

Q. How do you choose chunk size?

A. Big enough that a chunk is one coherent idea, small enough that its embedding is specific — a paragraph is a good default; tune to the documents and add ~15% overlap so boundary facts survive.

Q. Build-time vs query-time?

A. Build-time embeds and indexes the whole corpus once; query-time embeds one query, retrieves, and generates. Same embedding model must be used in both.

Grounding & agents

Q. How do you reduce hallucination in RAG?

A. Ground the prompt (answer only from passages, cite sources, allow ‘I don’t know’), keep top-k tight, and measure groundedness with a judge — retrieval quality is the real ceiling.

Q. What makes a RAG system 'agentic'?

A. The LLM decides how to retrieve — decomposing the question, retrieving iteratively, judging sufficiency, and searching again — instead of a single fixed retrieval.

Q. When is agentic RAG worth the cost?

A. On multi-hop / compound questions where one search misses evidence. On simple questions it just adds cost, so route by difficulty. Always weigh the measured quality gain against extra tokens.

Q. What is ReAct?

A. Reason + Act: an agent loop where the model interleaves reasoning with tool calls, observing each result before the next action, until it answers.

LangGraph

Q. What problem does LangGraph solve?

A. It turns hand-written agent control flow into a declared, drawable graph with state, nodes, and conditional edges — enabling checkpointing, streaming, parallelism, and human-in-the-loop without hiding the logic.

Q. State, node, edge, conditional edge — define each.

A. State: shared TypedDict of channels. Node: function returning a partial state update. Edge: fixed next step. Conditional edge: a router that reads state and returns the next node’s name.

Q. What is a reducer in LangGraph state?

A. An annotation (e.g. operator.add) that tells a channel to merge updates — appending — instead of overwriting; used to accumulate retrieved documents across loop iterations.

Q. How would you add human-in-the-loop?

A. Compile the graph with a checkpointer and interrupt before a sensitive node; the run pauses with its state persisted, a human approves/edits, then execution resumes.

Evaluation

Q. Why evaluate retrieval and generation separately?

A. They fail for different reasons: bad retrieval means the model never had the evidence; bad generation means it fumbled evidence it had. One combined score hides which half to fix.

Q. Explain hit@k, recall@k, MRR.

A. hit@k: any relevant doc in top-k. recall@k: fraction of relevant docs retrieved. MRR: reciprocal rank of the first relevant doc — rewards ranking it high.

Q. Pros and cons of LLM-as-judge?

A. Cheap, consistent, scalable, and correlates with human judgement — but carries judge bias, so trust it as a relative comparator (same judge/rubric) more than as an absolute score.

Security & production

Q. What is indirect prompt injection in RAG?

A. A retrieved document contains adversarial instructions the model might obey. Mitigate by treating retrieved text strictly as data, sandboxing it, and never letting it override the system prompt.

Q. How would you productionize this?

A. Hosted vector DB with an ANN index, a reranker, response + embedding caching, a difficulty router (naive vs agentic), eval in CI on a fixed question set, observability on retrieval quality, and injection defences.

13 · One-page cheat-sheet

The pipeline

```
ACQUIRE -> CHUNK -> EMBED -> INDEX  ||  RETRIEVE -> GENERATE
```

Say these numbers (this project)

Fact	Value
Corpus	2,012 arXiv papers -> 4,298 chunks
Embeddings	all-MiniLM-L6-v2, 384-dim, local/CPU
Index	FAISS IndexFlatIP (exact, cosine via normalized dot product)
Single-hop recall@5	1.00
Multi-hop recall@5	0.60 <- the gap the agent closes
Agent loop cap	MAX_AGENT_STEPS = 4
Tests	24 passed, keyless (LLM mocked)

Definitions in one line each

- **RAG** — retrieve docs, put them in the prompt, answer from them.
- **Embedding** — text as a vector where near = related.
- **Cosine sim** — angle between vectors; = dot product if normalized.
- **Chunk** — an idea-sized, metadata-tagged slice of a document.
- **ANN** — approximate nearest-neighbour; speed for a little recall.
- **Grounding** — answer only from sources; cite; allow abstaining.
- **Agentic RAG** — the LLM decides how to retrieve, iteratively.
- **ReAct** — reason + act loop with tools.
- **LangGraph** — agent as a state graph: nodes, edges, conditional edges.
- **Reducer** — a state channel that accumulates instead of overwriting.
- **hit@k / recall@k / MRR** — retrieval metrics against known-relevant docs.
- **LLM-as-judge** — grade answers for groundedness & correctness.

The 30-second project pitch

‘I built a research assistant over 2,000 arXiv papers with a fully-local retrieval pipeline — chunking, sentence-transformer embeddings, a FAISS index — and two answer paths: a single-shot RAG baseline and an agentic version that retrieves iteratively. I measured them: single-shot retrieval gets perfect recall on single-hop questions but only 0.60 on multi-hop ones, which is exactly the gap the agent is built to close. I then rebuilt the agent in LangGraph to show the same logic as a checkpointable graph. Every number reproduces offline; the LLM calls are cached and the whole thing is test-covered without a key.’

Companion to the project's `learning/` course (11 lessons) and the LangGraph branch notebook. Code references are live files in the repo — always be ready to open one.